17/10/22

Blind writes

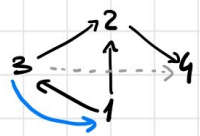
A write is blind if either
1. it is not the last write on a resource X
2. the following operation on X is a write

As long as blind writes are blind, we can move them around within the schedule without affecting reads-from / final-writes. We generate a new schedule S^{MOD} view equivalent to S

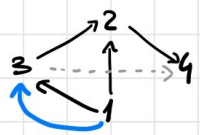
$y: r_1 \; w_3 \; w_1 \; w_2$
$x: r_2 \; r_3 \; w_4$
$z: r_3 \; w_3$

⇓

$y: r_1 \; w_1 \; w_3 \; w_2$
$x: r_2 \; r_3 \; w_4$
$z: r_3 \; w_3$

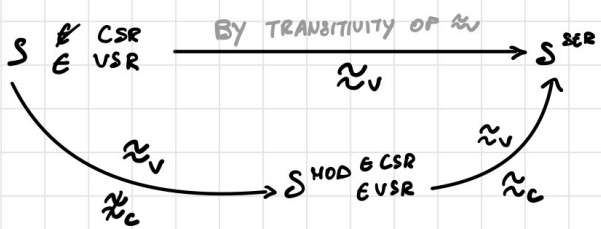


Transitive edges will be omitted



⇒ ACYCLIC! :)

What we did can be schematized by:



$\approx_V$ → View ser
$\approx_C$ → Conflict ser

Moving blind writes

$X: w_2 \; r_3 \; w_1 \; w_3 : S$

⇓

$X: w_1 \; w_2 \; r_3 \; w_3 : S^{MOD}$

Be careful when moving blind writes

S_4 :

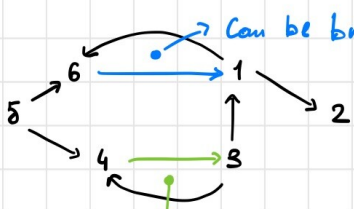
t :	r_6	r_5	w_6	w_1	w_2
x :	r_5	w_4	w_3	w_1	
y :	r_3	w_3	w_1	r_6	
z :	w_5	r_3	w_4	r_1	w_2

⇓ Swap $w_6(t) - w_1(t)$

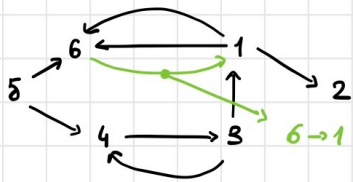
t :	r_6	r_5	w_1	w_6	w_2
⋮					

⇓

$S \notin CSR \wedge$
$S \notin VSR^*$



Can be broken by swapping $w_4 - w_3$



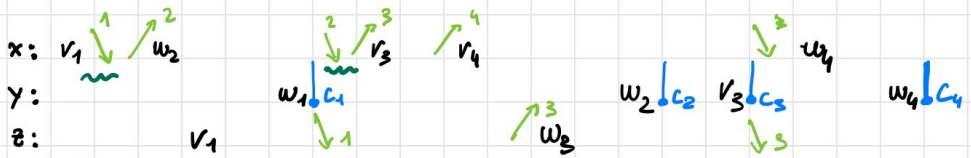
$6 \to 1$ is unavoidable

* see relation graph in your section

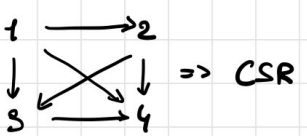
2PL Strict

$S \in VSR$. Check if $S \in CSR \wedge S \in 2PL(Strict)$

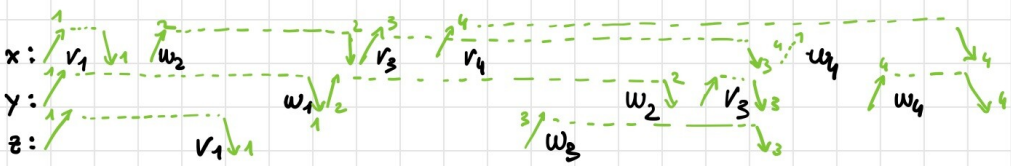
S : $r_1(x)$ $w_2(x)$ $r_1(z)$ $w_1(y)$ $r_3(x)$ $r_4(x)$ $w_3(z)$ $w_2(y)$ $r_3(y)$ $w_4(x)$ $w_4(y)$

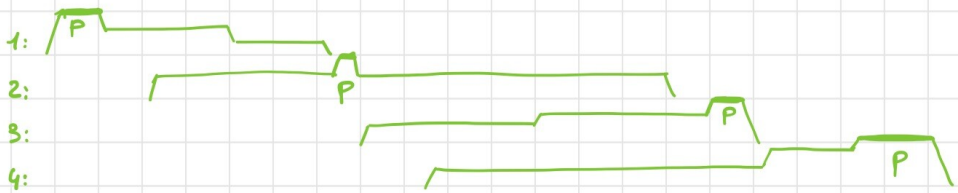


↳ NOT 2PL Strict



⇒ CSR





=> se 2PL

24/10/22

ES 1

	X	Y
$r_1(x)$	$UL_1(x)$	
$r_2(x)$	$SL_2(x)$	
$r_3(y)$		$UL_3(y)$
$w_3(y)$		$UL_3 \rightarrow XL_3(y)$
		$rel(XL_3)$
		$XL_2(y)$
	$rel(SL_2(x))$	
$w_1(x)$	$UL_1 \rightarrow XL_1(x)$	
$w_2(y)$	$rel(XL_1)$	
		$rel(XL_2)$

2PL? (with upgrade locks)

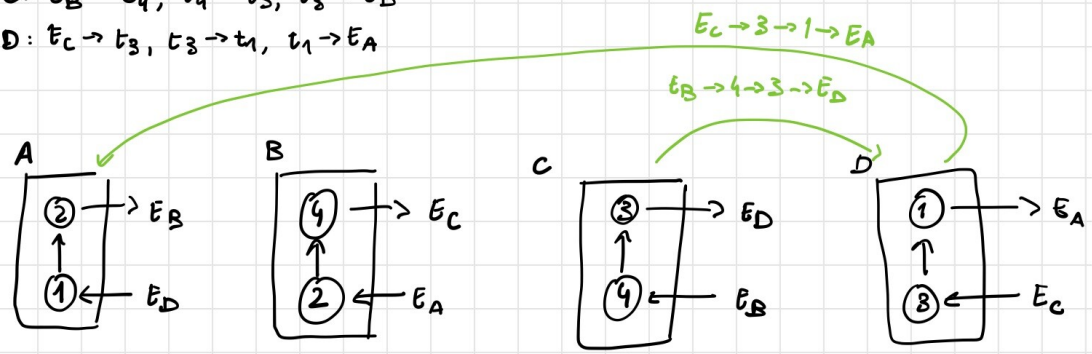
ES 2

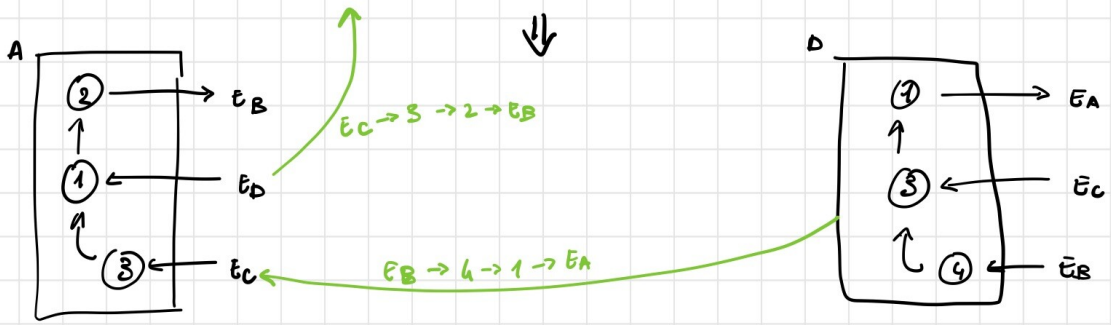
A: $E_D \rightarrow t_1$, $t_1 \rightarrow t_2$, $t_2 \rightarrow E_B$

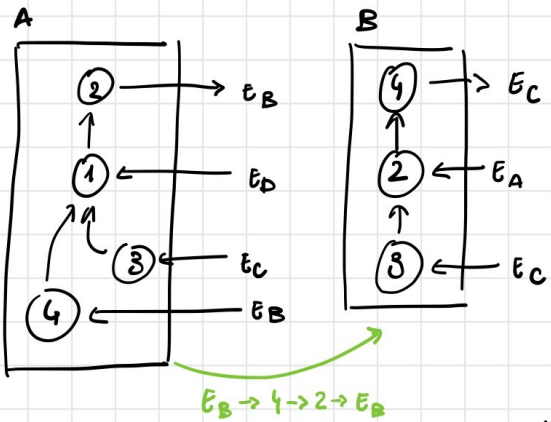
B: $E_A \rightarrow t_2$, $t_2 \rightarrow t_4$, $t_4 \rightarrow E_C$

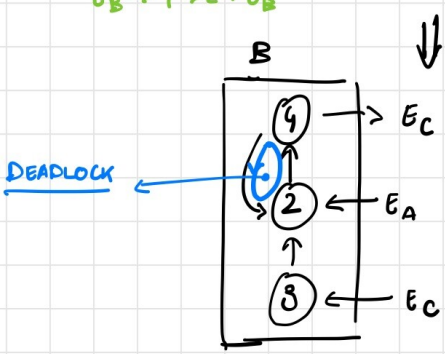
C: $E_B \rightarrow t_4$, $t_4 \rightarrow t_3$, $t_3 \rightarrow E_D$

D: $E_C \rightarrow t_3$, $t_3 \rightarrow t_1$, $t_1 \rightarrow E_A$









ES 4

X: r_4 r_2 w_4 w_3

Y: w_2 w_4 r_3

Z: w_4 r_3 r_8 r_8 w_6 w_9 r_5 r_{10}

TS? TS (MULTI)?

Quick way to check: look at the table and search for $.j$ $.i$ $(i<j)$. For TS (MULTI) ignore reads.

5/12/2022

ES1

Publications (PubCode, ISBN, Title, Date, ...)

- n = 1.000.000
- stored in a hash-based structure built on PubCode with
  - fill rate of 80%
  - 10 tuples/block
  - 1.11 avg lookup cost

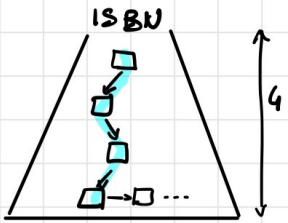
↳ UNIQUE

1) select * from Publications where ISBN = '00112233445' and (PubCode = 'ABC123' or PubCode = 'DEF456')

c = 1.11 + 1.11 = 2.22 I/O

('ABC123' , 'DEF456')

2) Let us add a secondary B+ index on ISBN with 4 levels and 8K leafs. Recalculate the cost of 1).



c = 4 + 1 = 5

(1 = data access)

ES2

Student (StudId, SSN, LastName, FirstName, City, Faculty)

- n = 20K
- 4 B+ indices with composite keys
  - I1: (StudId, SSN, Faculty)
  - I2: (LastName, FirstName, City)
  - I3: (Faculty, City, LastName)
  - I4: (Faculty, LastName, City)

With composite keys we first sort by K[0], then K[1]...

- Uniform distribution for non-unique attrs:
  - val(LastName) = 5000
  - val(FirstName) = 1000
  - val(City) = 1000
  - val(Faculty) = 10

1) Choose the best B+ index for:
select * from Student where City = 'Milan' and
Faculty = 'Computer Science'

I1, I2: not useful

I3: 20K / val(Faculty) = 2.000 t/Faculty
20K / val(City) = 20 t/City
$\rightarrow \frac{20K}{val(\cdot) \cdot val(\cdot)} = 2\,t$
=> 2t consecutive for each Faculty-City combination

I4: 2K t/Faculty, 2t with both desired attrs however not consecutive => worse than I3

ES3

T
- n = 1M in 100K b, entry sequenced
- B+ tree wit 3 lvls

1. Check that the avg fan-out is 100:

1M pointers => $x^3 \simeq 1M$ (3 = n lvls) => $x = 100$

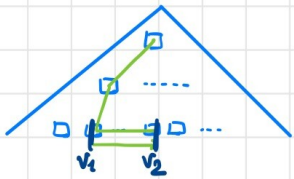
2. Avg sequential access is 10x faster than random. How many tuples be returned by an interval query on the primary key so that an execution plan is as fast as scanning the whole table?

$\frac{1M}{100K} = 10\,t$ -> block factor

$t_{RA} = 10\,t_{SA}$

P1: $1\,t_{RA} + (100K - 1)\,t_{SA} \simeq 100K\,t_{SA} \rightarrow 10K\,t_{RA}$

P2: $3\,t_{RA} + \underbrace{N/100}_{\text{leaf blocks accessed}}\,t_{RA} + N\,t_{RA}$ ← main structure accesses



$\Rightarrow \quad 10K\,t_{RA} = \underbrace{(3 + N/100 + N)}_{\approx N \text{ for } N \gg 300}\,t_{RA} \quad \Rightarrow \quad N = 10K$

ES4

Student (SNumber, Name, Surname, Address, Birth Date, ...)

- 64 Kt in 15KD in a primary hash on SNumber with fill factor ≤ 50%

Exam (SNumber, Course Code, Date, Grade)

- 180Kt in a primary B+ on SNumber with fan-out 22 and 10K leaf nodes

1) Calculate the cost of joining the two tables with the most convenient join method

Tree stats =
- 180Kt
- Fan out = 22
- Lvls = 4
- BF = 18 t/b
- $\frac{180K}{64K} = 2.8$ exam/student

Lvl	Nodes	Ptrs	Keys
1	1	22	21
2	22	22^2	22×21
3	22^2	22^3	$22^2 \times 21$
4	22^3	22^4	$22^3 \times 21$

$L \geq \#ts$

$\Rightarrow \sim \log_{22} \#ts$

S1: indexed N.L. with EXAM as external

$\Rightarrow$ SCAN: $3 + 10K \simeq 10K$

$\Rightarrow$ HASH ACC.: $180K \cdot 1 = 180K$

$\rightarrow 10K + 180K \cdot 1 = \underline{190K}$

OPTIMIZATION: since the leafs are ordered by SNumber, we can reuse the same tuple fetched from the hash, reducing hash lookups: $10K + 64K \cdot 1 = \underline{74K}$

S2: indexed N.L. with STUDENT as external

obviously worse since 1. hash has the best eq access
2. external table is larger.

2. Calculate the cost of the same operation considering that both tables use hash structures.

EXAM stats:
- 180K t $\vdash$ 12 exams/bucket => using 18 exams per block
- 15K b
- 66% F.F.

HASH JOIN = 15K + 15K = 30K